Reporte LeetCodes

García Rayón Dylan Osvaldo
Córdova García Diego
Robles de la Rosa Mauro Alexis

10 de noviembre de 2026

1. LeetCode 1011

Dada una lista de los pesos de varios paquetes y una cantidad de días fija, hallar la capacidad mínima de carga para entregarlos todos.

1.1. Resolución con Búsqueda Binaria

El problema plantea determinar la capacidad mínima de carga de un barco para trnasportar un cojunto secuencial de paquetes en un límite de D días, respetando estrictamente el orden original en el que se presentan.

El rango de capacidades posibles del barco está acotado de forma natural:

El límite inferior es el peso del paquete más pesado, ya que el barco debe ser capaz al menos de cargar el elemento individual de mayor masa.

El limite superior es la suma total de los pesos, que representa el escenario donde todos los paquetes se transportan en un solo día.

Existe una relación inversa, a mayor capacidad de carga, menor número de días necesarios para completar el envío; a menor capacidad, más días se requieren. La invariante del algoritmo (Búsqueda Binaria) garantiza que evaluando un punto medio mediante una simulación lineal, se puede descartar de manera segura la mitad del espacio de búsqueda hasta converger en la capacidad óptima mínima.

1.2. Optimización

Probar de manera secuencial cada capacidad entera desde los pesos máximos hasta la suma de los pesos tomaría un tiemppo proporcional al tamaño del rango total (O(Suma-Máximo)) lo cual es ineficiente si los pesos o la suma son grandes.

Al aplicar Búsqueda Binaria sobre el rango de capacidades, el espacio de búsqueda se reduce a la mitad en cada paso. Esto transforma un problema de optimización en uno de búsqueda logarítmica, reduciendo drásticamente el número de evaluaciones de capacidad de lineal a logarítmica con respecto al valor numérico de la suma de pesos.

1.3. Complejidad Temporal

O(n log S), donde n es el número de paquetes en el arreglo y S es la diferencia entre la suma total de los pesos y el peso máximo.

1.4. Complejidad Espacial

O(1). El algoritmo opera exclusivamente utilizando variables primitivas de control para los apuntadores del rango (low, high, mid) y contadores locales durante la simulación de los días, sin requerir estructuras de datos auxiliares ni memoria adicional.

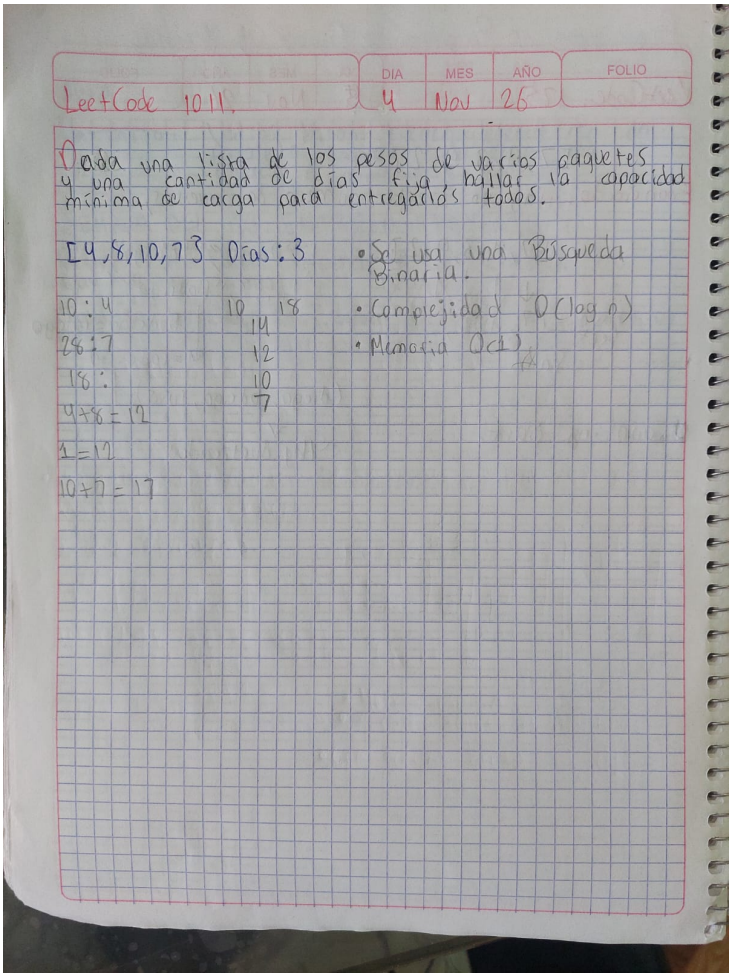


Figura 1: LeetCode 1011

2. LeetCode 98

Dada la raíz de un árbol binario determinar si es un árbol de búsqueda binario válido

2.1. Resolución con Atravesamiento de árboles (Preorden)

El problema consiste en determinar si un árbol binario dado es un Árbol Binario de Búsqueda (BST) válido. La condición fundamental de un BST dicta que para cualquier nodo, todos los valores en su subárbol izquierdo deben ser estrictamente menores que la clave del nodo, y todos los valores en su subárbol derecho deben ser estrictamente mayores.

Al recorrer el árbol en Preorden (visitando primero el nodo raíz y luego descendiendo hacía sus subárboles izq y der), la invariante clave que se propaga en cada llamada recursiva es un rango válido de valores permitidos.

Cuando se visita el nodo raíz, se valida inmediatamente si su valor se encuentra dentro de los límites actuales permitidos.

Al descender al subárbol izquierdo, el límite superior se actualiza al valor del nodo actual (ya que ningún descendiene izq puede ser mayor o igual al padre). Al descender al subárbol derecho, el límite inferior se actualiza al valor del nodo actual.

2.2. Optimización

Un error común de fuerza bruta consiste en verificar únicamente que nodo.izq.val ¡nodo.val ¡nodo.der.val para cada nodo de forma aislada. Esto es incorrecto porque un nodo en un subárbol izquierdo profundo podría violar la regla del BST con respecto a un ancestro superior (por ejemplo, ser mayor que la raíz del árbol).

Otra opción clásica es hacer un recorrido en orden y verificar que los elementos resultantes formen una secuencia estrictamente ascendente. Sin embargo, requiere almacenar o comparar el valor previo, mientras que el enfoque de Preorden con propagación de límites permite cortar la recursión de forma temprana tan pronto como se detecta una violación del rango, sin necesidad de recorrer todo el árbol si la condición falla en niveles superiores.

Con Preorden visita cada nodo exactamente una vez, aprovechando la pila de llamadas del sistema de manera óptima.

2.3. Complejidad Temporal

O(n), donde n es el número total de nodos en el árbol binario. En el peor de los casos, el algoritmo debe visitar cada nodo del árbol una sola vez para verificar y propagar los límites de validación, alcanzando una cota lineal estricta.

2.4. Complejidad Espacial

O(h), donde h es la altura del árbol binario.

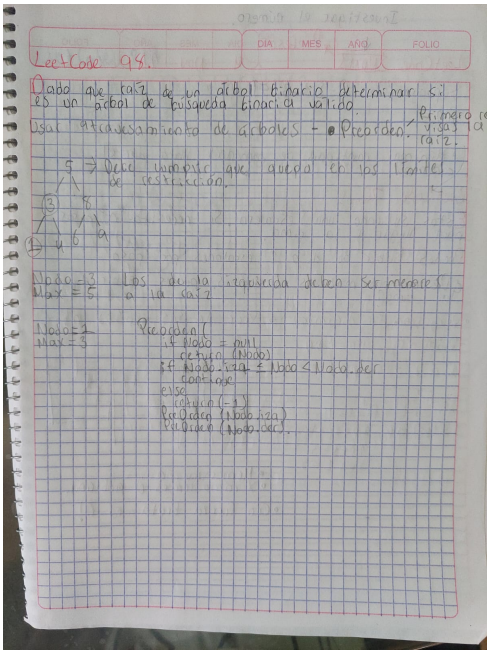


Figura 2: LeetCode 98

3. LeetCode 70

Estas subiendo una escalera. Se necesitan n pasos para llegar a la cima. Puedes subir de 1 o 2 escalones por paso. ¿De cuántas maneras distintas puedes llegar a la cima?

3.1. Resolución con Programación Dinámica

El problema plantea calcular de cuántas formas distintas se puede llegar a la cima de una escalera de n escalones, si en cada paso se permite avanzar opcionalmente de 1 o 2 escalones.

Para alcanzar un escalón i arbitrario, el último movimiento debió haber sido de 1 paso (viniendo desde el escalón i-1) o de 2 pasos (viniendo desde el escalón i-2).

La invariante de la programación dinámica establece que el número total de formas distintas para llegar al escalón i (definido como dp[i]) es la suma exacta de las formas de alcanzar los dos escalones previos: dp[i] = dp[i-1] + dp[i-2]

El comportamiento refleja exactamente la relación de recurrencia de la serie de Fibonacci, con casos base definidos para el escalón 1 (1 forma) y el escalón 2 (2 formas).

3.2. Optimización

Intentar resolver el problema por Fuerza Bruta mediante una función recursiva directa sin optimización evalúa repetidamente los mismos subproblemas múltiples veces. Esto provoca una explosión combinatoria que degrada el rendimiento a un costo exponencial de $O(2^n)$, volviéndose inviable para valores moderados de n.

Resolver el problema mediante Programación dinámica con Memorización (Top-Down) o iterativa (Bottom-Up) al almacenar los resultados de los subproblemas ya calculados, se evita cualquier recálculo redundante, reduciendo drásticamente el tiempo de ejecución a O(n).

Aunque una tabla de programación dinámica tradicional requeriría un arreglo auxiliar de tamaño n (O(n) de espacio), el análisis de la recurrencia demuestra que para calcular el estado actual solo se necesitan los dos valores inmediatamente anteriores (dp[i-1] y dp[i-2]). Por lo tanto, el uso de un par de variables escalares permite prescindir por completo del arreglo auxiliar, optimizando la memoria al límite óptimo.

3.3. Complejidad Temporal

O(n), donde n es el número total de escalones. El algoritmo procesa la solución mediante un único bucle iterativo que recorre linealmente desde el tercer escalón hasta n, realizando una cantidad constante de operaciones aritméticas elementales por cada iteración.

3.4. Complejidad Espacial

O(1). Al emplear una técnica de optimización de espacio basada en el almacenamiento exclusivo de las dos últimas variables de estado (descartando arreglos o tablas de tamaño n), el consumo de memoria auxiliar es completamente constante, independiente de la magnitud de la entrada.

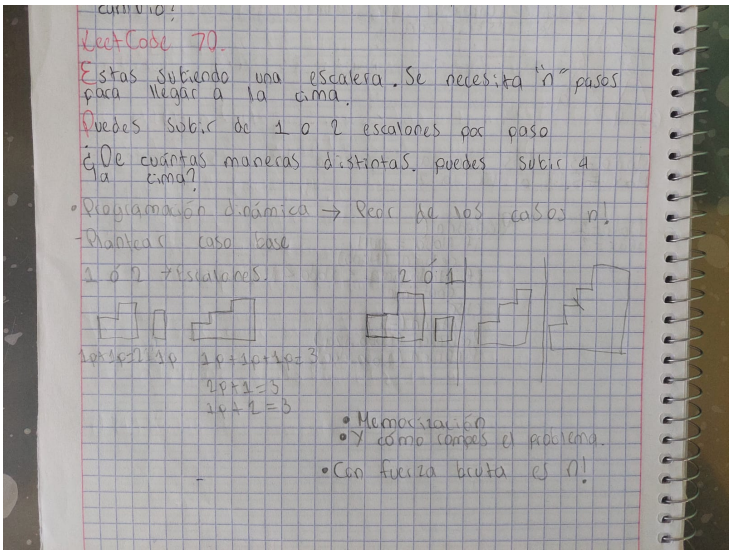


Figura 3: LeetCode 1011

4. Leet Code 210

Hay un total de ‘numCourses‘ cursos que debes tomar, etiquetados de 0 a ‘numCourses‘ - 1. Se da una matriz prerrequisites = [ai ; bi] indica que debes tomar el curso bi si quieres tomar el curso ai. Retorna el orden de cursos que tienes que tomar para acabar todos.

4.1. Resolución con Atravesamiento de grafos (BFS)

El problema consiste en encontrar el orden correcto en el que se deben tomar un conjunto de cursos dados ciertos prerrequisitos (dependencias dirigidas). Esto se modela como un problema de Ordenamiento Topológico en un Grafo Dirigido Acíclico.

Se procesa el grafo calculando inicialmente los grados de entrada de cada vértice (cantidad de dependencias pendientes que tiene cada curso).

La invariante central postula que cualquier nodo con un grado de entrada igual a 0 está libre de prerrequisitos y, por lo tanto, puede ser cursado y agregado a la secuencia de salida de forma inmediata.

Al extraer un nodo de la cola, se simula su completitud decrementando en uno el grado de entrada de todos sus vecinos adyacentes. Si el grado de un vecino llega a 0, se inserta en la cola para su procesamiento en el siguiente nivel. Si al finalizar el algoritmo el número de elementos procesados no coincide con el total de nodos, se determina la existencia de un ciclo, lo cual invalida el ordenamiento.

4.2. Optimización

Intentar probar todas las combinaciones posibles de secuencias de cursos (Fuerza Bruta) generaría una complejidad O(n!), volviéndose completamente intratable para números moderados de materias.

El uso de una cola combinada con el seguimiento de los grados de entrada (BFS) permite recorrer y ordenar el grafo de manera óptima y estrictamente lineal. A diferencia de las aproximaciones recursivas puras, el enfoque iterativo con BFS evita los riesgos de

desbordamiento de la pila de llamadas en grafos profundos. Permite validar de manera simultánea la existencia de ciclos en el grafo sin requerir algoritmos de detección adicionales complejos.

4.3. Complejidad Temporal

O(V + E), donde V es el número de vértices (cursos) y E es el número de aristas (prerrequisitos). El cálculo inicial de los grados de entrada y la construcción de la lista de adyacencia recorren todas las aristas y vértices una vez. Posteriormente, cada vértice entra y sale de la cola BFS exactamente una vez, y cada arista se recorre una sola vez cuando se actualizan los grados de los vecinos.

4.4. Complejidad Espacial

O(V + E). Se requiere memoria auxiliar para almacenar la estructura del grafo (lista de adyacencia de tamaño V + E), el arreglo unidimensional para los grados de entrada de tamaño V, la cola de procesamiento BFS y el arreglo de resultados finales, los cuales escalan de forma lineal respecto a las dimensiones de la entrada.

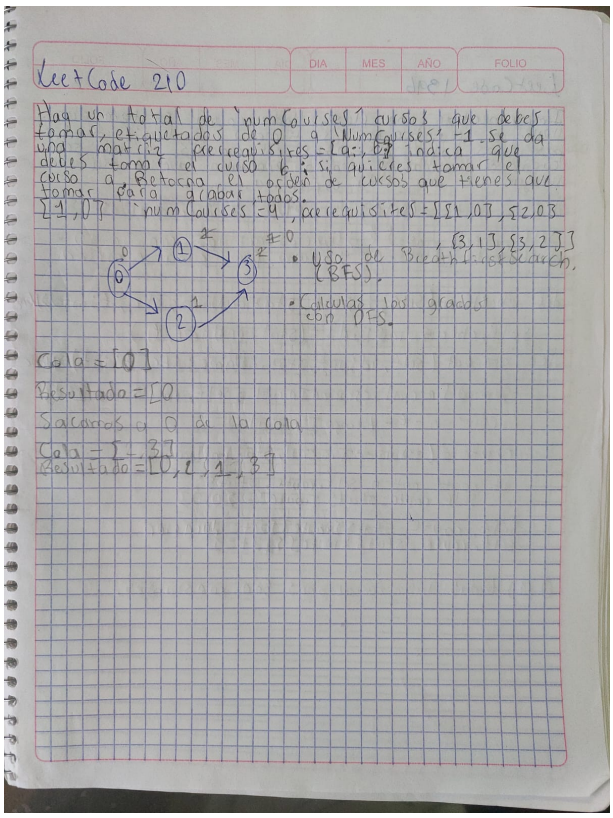


Figura 4: LeetCode 1011